Eniac Timesheet Platform

Technical Architecture, API, Data Model,
Security & Operational Documentation

1. Executive Summary

Eniac is a web-based workforce/project operations platform centered on project administration, employee and supervisor management, weekly timesheets, timesheet approvals, project documents, notifications, activity history, and administrative reporting. The repository contains a React/Vite single-page frontend and an Express/TypeScript backend backed by MongoDB, with Vercel deployment configuration and Vercel Blob integration for project documents.

The platform uses a role model with two persisted roles (admin and user) plus a supervisor capability flag. Authorization is implemented both in the frontend for navigation/UX and, more importantly, in backend middleware and service-level checks. Timesheets follow a draft → pending → approved/declined/withdrawn workflow and generate notifications/activity records as state changes.

Overall assessment
The codebase is a credible functional MVP with a reasonably clean separation between routes, controllers, services, middleware, and persistence. The main production risks are not architectural complexity but contract drift and security/operational gaps: refresh-token rotation is incomplete, some authorization rules are inconsistent between layers, notification ownership checks are missing, document upload/delete contracts are mismatched between frontend and backend, and several query parameters advertised by the frontend are ignored by backend list endpoints.

Key technologies

Layer	Technology	Observed responsibility
Frontend	React 19 + TypeScript + Vite	SPA UI, routing, role-specific pages, local state/context, API integration
Styling/UI	Tailwind CSS + custom UI components	Responsive application shell, forms, tables, cards, status indicators
Charts	Recharts	Administrative reporting/dashboard visualizations
Backend	Node.js + Express 4 + TypeScript	REST API, authentication, authorization, business logic
Validation	Zod	Request payload validation for major write operations
Database	MongoDB	Users, projects, timesheets, notifications, activities, documents, sessions
Auth	jose + bcryptjs	HS256 JWT access/refresh tokens and password hashing
File storage	Vercel Blob	Project document object storage
Security middleware	Helmet, CORS, express-rate-limit	HTTP headers, origin policy, rate limiting
Testing	Vitest + Supertest	Backend auth/access/security/timesheet tests
Deployment	Vercel	Frontend static SPA and backend serverless entrypoint configuration

2. Scope, Evidence & Analysis Method

This document is based on a review of the repository and focuses on functionality that is actually present in the codebase. The assessment reflects the current implementation rather than assumptions about how the application is intended to work. Where something is incomplete, inconsistent, or appears to be left over from an earlier version, it is called out separately as a finding.

The review covered the main parts of both the frontend and backend, including:

- Frontend routing, application contexts, service clients, domain types, utilities, pages, and shared UI components.
- Backend application setup, API routes, controllers, services, middleware, validation schemas, persistence helpers, and tests.
- Package manifests, TypeScript and Vite configuration, environment configuration, and Vercel deployment configuration.
- Database collection indexes and the main application workflows, including authentication, users, projects, timesheets, approvals, documents, notifications, activities, and reporting.

The source code reviewed consisted of approximately **5,092 backend lines** and **11,516 frontend lines**, excluding dependencies and generated source maps.

Important distinction
The repository contains both production-looking API/service code and several legacy/demo artifacts (mock data, demo login helpers, compiled test files, and duplicate/experimental routes). Those artifacts should be treated as technical debt unless they are intentionally retained for development/demo purposes.

3. System Architecture

3.1 Logical architecture

```
Browser / React SPA
        |
        | HTTPS / REST JSON (Bearer access token + refresh cookie on auth)
        v
Express API (/api/v1)
  ├─ Auth middleware / role checks
  ├─ Resource access middleware
  ├─ Controllers
  ├─ Services / business rules
  └─ MongoDB persistence
        |
        +--> MongoDB collections
        |
        +--> Vercel Blob (project documents)
        |
        +--> Notifications / Activity records
```

The backend follows a layered structure: route modules compose middleware and controllers; controllers parse requests and translate service outcomes into HTTP responses; services own business rules and MongoDB operations; shared middleware handles authentication and authorization; persistence helpers provide database access and indexes.

3.2 Frontend architecture

- React Router defines separate route spaces for admin, ordinary user, and supervisor experiences.
- AuthContext owns authentication state and current-user initialization.
- AppDataContext acts as a broad client-side data facade for projects, users, timesheets, notifications, activities, and documents.
- NotificationContext and ToastContext provide cross-cutting UI behavior.
- Service modules isolate REST calls from page components.
- A reusable component library supplies tables, cards, inputs, modals, badges, loading/error states, navigation, and layout.

3.3 Backend architecture

- app.ts configures CORS, Helmet, JSON/urlencoded parsing, global rate limiting, route registration, and error handling.
- server.ts loads environment variables, attempts index creation, then starts the HTTP server.
- MongoDB connection state is cached on globalThis, which helps avoid repeated connections in serverless/hot-reload scenarios.
- Business workflows emit secondary side effects such as notifications and activity records.

3.4 Deployment topology

Component	Deployment evidence	Notes
Frontend	frontend/vercel.json	SPA rewrite sends all paths to index.html
Backend	backend/vercel.json	Vercel Node build for api/index.ts; /api/* routed to that entrypoint
Database	MongoDB URI env var	MongoDB Atlas-style example is supplied
Blob storage	BLOB_READ_WRITE_TOKEN + @vercel/blob	Public Blob objects are created by upload controller
Cron	CRON_SECRET + /notifications/cron/deadline	Endpoint exists; external scheduler configuration is not included in repository

4. Frontend Technical Design

4.1 Route map

Route area	Representative routes	Access
/admin	/admin/dashboard, /projects, /users, /supervisors, /timesheets, /approvals, /reports, /settings	role=admin
/user	/user/dashboard, /projects, /timesheets, /submissions, /notifications, /settings	role=user
/supervisor	/supervisor/timesheets, /supervisor/approvals	role=user + isSupervisor=true
Authentication	/adminlog, /userlog	Public

4.2 State and data flow

```
Page
  -> Context action
     -> Service module
        -> apiClient
           -> fetch(/api/v1/...)
              -> Express route
                 -> middleware
                    -> controller
                       -> service
                          -> MongoDB / Blob
                       <- response
                    <- JSON
           <- typed response
     <- context state update
  <- page re-render
```

AppDataContext initially loads projects, users, timesheets, activities, and documents in parallel after authentication. Notifications are refreshed through a separate context/service path. Mutations generally update the corresponding context array after a successful service call.

4.3 Authentication client behavior

- The access token is stored in localStorage under eniac_access_token.
- The browser sends the token as Authorization: Bearer <token>.
- credentials: include is enabled so the refresh cookie can accompany requests.
- The client does not implement a refresh-token endpoint because the backend exposes no refresh route; it simply retries one safe request once and removes the access token on 401.

4.4 Frontend strengths

- Clear separation of route/page/service/context responsibilities.
- Strong TypeScript settings in the application tsconfig, including noUnusedLocals and noUnusedParameters.
- Role-aware navigation and route protection reduce accidental exposure of UI capabilities.
- Reusable UI primitives make the interface maintainable compared with page-specific markup everywhere.

4.5 Frontend risks

- Frontend permission helpers read projects/users from localStorage, while the authoritative backend permissions read MongoDB. This creates two sources of truth.
- AppDataContext catches and suppresses several timesheet errors by returning undefined, which can make operational failures appear as no-op UI behavior.
- The document service sends JSON-shaped document metadata for createDocument, while the backend upload endpoint requires multipart/form-data with a file. The intended upload path is therefore not aligned.
- The frontend Document type uses size:string/type/uploadedAt, whereas backend documents use numeric size/mimeType/createdAt plus storage metadata.
- The notification type union omits 'document', even though the backend emits a notification with type 'document'.
- Some service methods pass query parameters such as userId, supervisorId, weekStart that the backend listTimesheets controller does not currently consume.

5. Backend Technical Design

5.1 API conventions

- Base API prefix: /api/v1
- JSON response envelope commonly uses { resource } or { data }.

- Errors commonly use { error: { code, message } }.
- HTTP 401 is used for missing/invalid authentication; 403 for authorization failures; 404 for missing resources; 409 for user uniqueness conflicts; 400 for many validation/business-rule errors; 500 for server errors.

5.2 Endpoint catalogue

Method	Path	Authorization	Purpose
POST	/auth/login	Public	Authenticate user; returns user + access token; sets refresh cookie
GET	/auth/me	Authenticated	Return current user
POST	/auth/logout	Authenticated	Delete current refresh session/cookie
POST	/auth/forgot-password	Public	501 — not implemented
POST	/auth/reset-password	Public	501 — not implemented
GET	/users	Authenticated	List users
GET	/users/:id	Admin	Get user
POST	/users	Admin	Create user
PATCH	/users/:id	Admin	Update user
POST	/users/:id/deactivate	Admin	Deactivate
POST	/users/:id/activate	Admin	Activate
PATCH	/users/:id/supervisor	Admin	Assign/remove supervisor
GET	/supervisors	Authenticated	List active supervisors
GET	/supervisors/:id/users	Authenticated	List supervised users
GET	/projects	Authenticated	List projects for current role
GET	/projects/:id	Project access	Get project
POST	/projects	Admin	Create project
PATCH	/projects/:id	Admin + project edit	Update project
DELETE	/projects/:id	Admin + project edit	Delete project
POST	/projects/:id/team	Admin + project access	Add member
DELETE	/projects/:id/team/:userId	Admin + project access	Remove member
PATCH	/projects/:id/supervisor	Admin + project edit	Assign supervisor
GET	/projects/:projectId/documents	Project access	List project documents
POST	/projects/:projectId/documents	Project access	Multipart file upload
DELETE	/projects/:projectId/documents/:documentId	Project access	Delete document
GET	/timesheets	Authenticated	List role-scoped timesheets
GET	/timesheets/:id	Timesheet access middleware + owner/admin controller check	Get timesheet
POST	/timesheets	Authenticated	Create draft timesheet for current user
PATCH	/timesheets/:id	Timesheet edit	Update draft/declined/withdrawn timesheet
POST	/timesheets/:id/submit	Timesheet edit	Submit for review
POST	/timesheets/:id/withdraw	Timesheet edit	Withdraw pending submission
GET	/approvals	Authenticated	List pending approvals; scoped for supervisors
POST	/approvals/:id/approve	Review access	Approve pending timesheet
POST	/approvals/:id/decline	Review access	Decline with reason
GET	/notifications	Authenticated	List current user's notifications
GET	/notifications/unread-count	Authenticated	Compute unread count
POST	/notifications	Authenticated	Create notification
POST	/notifications/:id/read	Authenticated	Mark notification read
POST	/notifications/read-all	Authenticated	Mark current user's notifications read
POST	/notifications/cron/deadline	CRON_SECRET	Create deadline notifications
GET	/activities	Authenticated	List all activities
GET	/activities/users/:userId	Authenticated	User activity list
GET	/activities/projects/:projectId	Authenticated	Project activity list
GET	/activities/timesheets/:timesheetId	Authenticated	Timesheet activity list
POST	/activities	Authenticated	Create activity
GET	/reports/hours-by-project	Admin	Aggregate hours by project
GET	/reports/hours-by-employee	Admin	Aggregate hours by employee
GET	/reports/overtime	Admin	Aggregate regular/overtime totals

GET	/reports/timesheet-status	Admin	Status breakdown

6. Data Model & Persistence

MongoDB is used without a formal ODM schema layer. Application-level interfaces and Zod schemas provide most type/validation guarantees, while MongoDB indexes provide important uniqueness and query performance constraints.

6.1 Collections

Collection	Primary fields	Important indexes
users	_id, name, email, employeeId, department, role, isSupervisor, status, supervisorId, passwordHash, timestamps	email unique; employeeId unique; supervisorId; status; isSupervisor
projects	_id, name, sowNumber, client, description, dates, status, managerId, supervisorId, teamMemberIds, documentIds, timestamps	sowNumber unique; status; managerId; supervisorId; teamMemberIds; date fields
timesheets	_id, userId, projectId, weekStart, entries, notes, regularHours, overtimeHours, totalHours, status, submittedAt, review, timestamps	userId; projectId; weekStart; status; unique userId+projectId+weekStart
notifications	_id, userId, type, title, message, read, relatedId, createdAt	userId; read; createdAt desc
activities	_id, userId, projectId, timesheetId, description, createdAt	userId; projectId; timesheetId; createdAt desc
documents	_id, projectId, name, size, mimeType, storageKey, url, uploadedBy, createdAt	projectId; uploadedBy; createdAt desc
sessions	_id, userId, refreshToken, expiresAt, createdAt	userId; expiresAt TTL

6.2 Relationships

```
User 1 ---- * Timesheet
User 1 ---- * Activity
User 1 ---- * Notification
User 1 ---- * Session
User 1 ---- * Document (uploadedBy)

Project 1 ---- * Timesheet
Project 1 ---- * Document
Project 1 ---- * Activity

Project.managerId       -> User
Project.supervisorId    -> User
Project.teamMemberIds[]  -> User
Timesheet.userId         -> User
Timesheet.projectId      -> Project
```

```
Timesheet.review.reviewedBy -> User (logical reference)
```

These are logical application references, not MongoDB foreign keys. Deleting a project does not show a repository-level cascade for its timesheets/documents/activities, so referential cleanup is an operational concern.

7. Core Business Workflows

7.1 Authentication

1. Client submits email/password to POST /auth/login.
2. Backend validates the payload with Zod.
3. User is looked up by lowercased email and active status.
4. bcrypt compares the supplied password with passwordHash.
5. Backend signs a 1-hour HS256 access token and a 7-day refresh token.
6. Refresh token is stored in the sessions collection and also placed in an HttpOnly cookie.
7. Access token is returned to the frontend and persisted in localStorage.

7.2 Project lifecycle

- Admin creates project with manager, supervisor, and at least one team member.
- Project status supports draft, active, completed, overdue, archived.
- Project access is granted to admins, team members, and supervisors assigned to the project.
- Project editing is intended for admins and the project supervisor, although route middleware also contains an admin-only requirement that currently makes the supervisor edit branch unreachable for non-admin users.

7.3 Timesheet lifecycle

```
draft
  | submit
  v
pending ---- withdraw ----> withdrawn
  |
  +---- approve ----> approved
  |
  +---- decline ----> declined
                        |
                        +---- submit ----> pending

declined/withdrawn
  |
  +---- edit ----> draft-like editable state
  +---- submit --> pending
```

The service enforces Monday normalization for weekStart. Regular entries are Monday–Friday; overtime entries are Saturday–Sunday. Each individual day is capped at 24 hours. Descriptions are required for entries containing hours. Totals are recalculated server-side.

7.4 Approval behavior

- Admins can review all pending timesheets.
- Supervisors can review timesheets associated with supervised projects/users according to approval.service rules.
- Approval records reviewer and timestamp; decline additionally records a reason.
- Approval/decline creates a notification for the timesheet owner and an activity record.

7.5 Documents

- Backend uses multer memoryStorage, validates MIME type and size (10 MB max), sanitizes the original filename, and uploads to Vercel Blob.

- Allowed file types include PDF, common images, Word, Excel, plain text, and CSV.
- Blob access is configured as public, so the returned URL is directly usable if otherwise undisclosed.
- The metadata is stored in MongoDB and an activity is emitted.

8. Security Architecture & Findings

8.1 Existing controls

- Helmet is enabled.
- CORS is restricted to FRONTEND_URL and credentials are enabled.
- Global and authentication-specific rate limits are configured.
- Passwords are hashed with bcryptjs at 12 salt rounds.
- Access tokens are short-lived (1 hour).
- Refresh tokens are stored in an HttpOnly cookie and sessions have a MongoDB TTL index.
- Backend authorization checks are based on the authenticated user's role, supervisor capability, ownership, and project relationships.
- Zod validates major write payloads.

8.2 High-priority security/authorization findings

Priority	Finding	Evidence/impact	Recommendation
Critical/High	Notification ID ownership is not checked	POST /notifications/:id/read calls markNotificationAsRead(id) without constraining userId. An authenticated user who knows another notification ID could mark it read.	Update query to {_id, userId}; add authorization test.
High	Activity endpoints are broadly readable/creatable	All activity routes require authentication but do not enforce ownership/project access. POST /activities accepts caller-supplied userId/projectId/timesheetId.	Scope reads to authorized resources and derive actor userId from req.user.
High	Document delete is not ownership/role restricted	Project access is enough to call deleteDocument; service deletes the Blob and Mongo record without checking uploader/admin/supervisor policy.	Define document deletion policy and enforce it server-side.
High	Public Blob objects	uploadDocument uses access:'public'. Any holder of a Blob URL may access the object.	Prefer private storage or signed URLs if documents are sensitive.
High	Refresh-token rotation endpoint absent	Service supports refreshUserSession but no route/controller exposes it. Access tokens therefore expire without a functional browser refresh flow.	Add POST /auth/refresh, rotate refresh tokens, revoke old sessions.
Medium	JWT secret fallback	jwt.ts falls back to 'change-me-in-production' if JWT_SECRET is missing, while env validation is not called at import time.	Fail closed at startup and never use a production fallback secret.
Medium	CRON secret fallback	Deadline endpoint falls back to a known placeholder when CRON_SECRET is missing.	Require CRON_SECRET in production.
Medium	Error handling leaks internal messages	Several 500 responses return err.message directly.	Log server details, return stable generic messages to clients.

9. Engineering Quality & Consistency Review

9.1 Contract mismatches

Area	Observed mismatch	Impact
Documents	Frontend createDocument sends JSON metadata; backend requires multipart file upload.	Upload functionality is likely broken when using the current service directly.
Document model	Frontend uses size:string/type/uploadedAt; backend uses size:number/mimeType/createdAt/storageKey/url.	Type/serialization mismatch and UI bugs likely.
Notifications	Frontend union omits 'document'; backend emits type 'document'.	Type-level and rendering assumptions can diverge.
Timesheet filters	Frontend exposes getTimesheetsByUserId/supervisorId/current-week query parameters; backend list controller only consumes projectId/status and builds role scope itself.	Some client filters are silently ignored.
Refresh auth	Service contains refreshUserSession, but no route/controller/client refresh method exists.	Token lifecycle is incomplete.
Project edit authorization	projects route applies requireAdmin before requireProjectEdit.	Non-admin supervisors cannot reach the supervisor edit branch even though canEditProject permits them.

9.2 Authorization inconsistencies

- timesheets/:id has requireTimesheetAccess middleware that allows a project supervisor to access a timesheet, but the controller then performs an additional check allowing only admin or owner. This can reject a supervisor after middleware has granted access.
- approval.service.canReviewTimesheet allows a supervisor when they are project supervisor, supervisor of the user, or a team member on the project. middleware/access.ts canReviewTimesheet only allows project supervisor (or admin). The two policies differ.
- Frontend canReviewTimesheet uses localStorage-derived supervised projects/users, while backend approval rules use MongoDB.

9.3 Data integrity concerns

- MongoDB references are stored as ObjectIds but there are no database-level foreign keys; deletion and reassignment must be managed by service logic.
- Some service operations generate activity/notification side effects after the primary write without a transaction. If a secondary write fails, the primary business state may still have changed.
- Timesheet creation performs a pre-check for an existing record and the collection also has a unique compound index. The index protects against races, but duplicate-key errors are not explicitly translated into a friendly conflict response.
- Project update does not show the same cross-field validation guarantees when only one of startDate/endDate/deadline is changed; partial updates can therefore deserve an invariant revalidation strategy.

9.4 Maintainability observations

- There is a generally understandable service/controller separation.
- Some controllers import middleware functions that are not used, indicating cleanup opportunities.
- There are duplicate/experimental test files and route files (for example users-copy.ts and several test-* files) in the backend tree.
- Mock datasets remain in frontend/src/mock, suggesting an earlier local/mock mode or demo path that should be isolated from production code.
- The root package.json contains dependencies that overlap with backend tooling while the actual backend/frontend packages are maintained separately; workspace/package ownership could be clarified.

10. Testing Strategy & Coverage

The repository includes four principal backend test suites: auth.test.ts, access.test.ts, security.test.ts, and timesheet.test.ts.

Test area	Observed coverage
Authentication	Login and JWT/authentication-related behavior; mocks for database/JWT
Authorization	Project and timesheet access/edit/review helper functions
Security	Role rejection, user access isolation, approval rejection, supervisor restrictions
Timesheets	Business rules around timesheet lifecycle and validation

Recommended missing tests

- Notification ownership: user A cannot mark user B's notification as read.
- Activity authorization: user cannot create an activity attributed to another user.
- Document upload: MIME/size validation, Blob failure behavior, deletion authorization, and project isolation.
- Supervisor access contract: GET timesheet and approval endpoints should share one explicit policy.
- Refresh flow: refresh token validation, rotation, revocation, expired session, and inactive-user behavior.
- Project deletion: verify whether dependent timesheets/documents/activities are retained, blocked, or cascaded.
- API contract tests for frontend/backend document payloads and all query filters used by service clients.
- Rate-limit behavior and CORS/credentials behavior in production-like configuration.

11. Configuration, Deployment & Operations

11.1 Environment variables

Variable	Purpose	Required/observed
MONGODB_URI	MongoDB connection string	Required
MONGODB_DB_NAME	Database name	Required
JWT_SECRET	HS256 signing secret	Required; must not use fallback
CRON_SECRET	Authorization for deadline cron endpoint	Required in production
BLOB_READ_WRITE_TOKEN	Vercel Blob access	Required for document upload
PORT	HTTP port	Optional; defaults 3001
FRONTEND_URL	Allowed CORS origin	Optional; defaults localhost:5173
NODE_ENV	Controls production behavior/rate limits/cookie secure flag	Runtime setting
VITE_API_BASE_URL	Frontend API base URL	Optional; defaults /api/v1

11.2 Local development flow

```
# backend
cd backend
npm install
npm run dev

# frontend
cd frontend
npm install
npm run dev

# backend tests
cd backend
npm test

# production build checks
```

```
cd backend
npm run build

cd frontend
npm run build
```

The frontend Vite dev server proxies /api/v1 to http://localhost:3001. In deployment, VITE_API_BASE_URL should point to the deployed API if the frontend and API are not served under a compatible origin.

11.3 Operational recommendations

- Use a secret manager/platform environment configuration rather than checked-in secrets.
- Monitor MongoDB connection failures, Blob upload failures, 401/403/429 rates, and 5xx responses.
- Add structured request IDs/correlation IDs to connect frontend failures to backend logs.
- Make deadline notification cron idempotent; the current job can send repeated notifications whenever invoked within the three-day window.
- Add database backup/restore procedures and define retention for activities, notifications, sessions, and documents.
- Document the production domains, CORS policy, cookie behavior, and Vercel project-to-environment mapping.

12. Representative API Contracts

12.1 Login

```
POST /api/v1/auth/login
Content-Type: application/json

{
  "email": "employee@example.com",
  "password": "Password123"
}

200
{
  "user": {
    "id": "...",
    "name": "...",
    "email": "...",
    "employeeId": "...",
    "department": "...",
    "role": "admin|user",
    "isSupervisor": true,
    "status": "active",
    "supervisorId": "..."
  },
  "accessToken": "..."
}

Set-Cookie: refreshToken=<HttpOnly token>; Path=/api/v1/auth
```

12.2 Timesheet create

```
POST /api/v1/timesheets

{
  "projectId": "<ObjectId>",
```

```
  "weekStart": "2026-09-07",
  "entries": [
    {
      "id": "entry-1",
      "description": "Implementation",
      "entryType": "regular",
      "hours": {
        "mon": 8, "tue": 8, "wed": 8, "thu": 8,
        "fri": 8, "sat": 0, "sun": 0
      }
    }
  ],
  "notes": "Weekly work"
}

201
{
  "timesheet": {
    "status": "draft",
    "regularHours": 40,
    "overtimeHours": 0,
    "totalHours": 40
  }
}
```

12.3 Document upload (backend contract)

```
POST /api/v1/projects/:projectId/documents
Content-Type: multipart/form-data

file=<binary>

Allowed MIME types:
PDF, PNG, JPEG, GIF, WEBP, DOC, DOCX, XLS, XLSX, TXT, CSV

Maximum size: 10 MB
```

This is the contract the frontend document service should be changed to implement.

13. Recommended Engineering Roadmap

Phase	Priority	Work	Outcome
0 — Stabilize	P0	Fix notification ownership, activity attribution/access, document deletion policy, JWT/cron fallbacks, and 500-message leakage.	Closes immediate security and operational gaps.
1 — Contract alignment	P0	Unify frontend/backend document types and multipart upload; add notification type; align timesheet filters and response types.	Removes client/server drift.
2 — Authorization	P0	Create one canonical authorization policy for projects/timesheets/approvals and use it from middleware + services + frontend capability checks.	Predictable access control.

3 — Authentication lifecycle	P1	Expose refresh endpoint, rotate refresh tokens, revoke old sessions, handle 401 refresh in apiClient.	Reliable long-lived sessions.
4 — Data integrity	P1	Define deletion/cascade policy; add transactions or compensating behavior for primary write + notification/activity side effects.	More consistent state.
5 — Testing	P1	Add API contract, security, document, refresh, notification ownership, and supervisor-policy tests.	Higher regression confidence.
6 — Observability	P1	Structured logs, correlation IDs, metrics, error monitoring, cron execution logs.	Faster diagnosis and operations.
7 — Architecture cleanup	P2	Remove duplicate test/route artifacts, isolate demo/mock data, clarify package/workspace ownership.	Lower maintenance burden.
8 — Production hardening	P2	Private/signed document URLs, CSP tuning, secret rotation, backups, retention, rate-limit tuning, deployment runbook.	Production-ready posture.

14. Codebase Map

Path	Role
frontend/src/App.tsx	Top-level route tree and role-protected navigation
frontend/src/main.tsx	React root and provider composition
frontend/src/contexts/AuthContext.tsx	Current-user/auth state
frontend/src/contexts/AppDataContext.tsx	Application data facade and client cache
frontend/src/services/apiClient.ts	Fetch wrapper, auth header, timeout, basic retry/error handling
frontend/src/services/*.ts	Domain-specific REST client functions
frontend/src/pages/**	Admin, user, supervisor, and authentication screens
frontend/src/components/**	Reusable UI, layout, approval, dashboard, and project components
backend/src/app.ts	Express app construction and global middleware
backend/src/server.ts	Runtime bootstrap and index initialization
backend/src/routes/*.ts	REST endpoint registration and middleware composition
backend/src/controllers/*.ts	HTTP request/response translation
backend/src/services/*.ts	Business logic and persistence operations
backend/src/middleware/auth.ts	JWT authentication and role gates
backend/src/middleware/access.ts	Resource-level access rules
backend/src/schemas/*.ts	Zod request schemas
backend/src/lib/mongodb.ts	MongoDB connection lifecycle
backend/src/lib/collections.ts	Collection names and indexes
backend/src/lib/jwt.ts	Access/refresh JWT creation and verification
backend/src/tests/*.test.ts	Backend automated tests
backend/src/scripts/**	Demo/seed data

15. Functional Capability Summary

Capability	Implemented behavior
Identity & access	Admin/user roles, supervisor flag, active/inactive users, supervisor assignments
Project management	Create/update/delete, status, manager/supervisor, team membership
Timesheet capture	Weekly entries, regular/overtime rules, notes, totals
Submission workflow	Draft, submit, withdraw, approve, decline
Notifications	User notifications, read state, assignment/submission/review/deadline events
Activity history	User/project/timesheet activity records

Documents	Project-scoped upload/list/delete using Vercel Blob + MongoDB metadata
Reporting	Hours by project, hours by employee, overtime totals, status breakdown
Dashboards	Admin/user dashboards and reusable stat/deadline/activity components
Settings	Admin and user settings screens exist; exact persisted setting scope should be validated separately
Password recovery	Routes exist but explicitly return 501/not implemented

16. Nonfunctional Assessment

Dimension	Assessment	Notes
Security	Moderate, needs hardening	Good baseline middleware/password hashing/role checks; several authorization gaps remain.
Scalability	Moderate for MVP	MongoDB indexes and cached connection help; several list endpoints return unpaginated datasets.
Performance	Moderate	Parallel initial frontend fetches; some notification/unread and supervisor logic performs repeated queries.
Reliability	Moderate	Business writes and side effects are not transactional; error handling is inconsistent.
Maintainability	Moderate	Layering is good; duplicate/demo artifacts and contract drift increase risk.
Testability	Moderate	Useful backend unit/security tests exist; frontend and API contract coverage is limited.
Observability	Basic	Pino exists, but error paths still use console.error and there is no evident request correlation/metrics layer.
Deployment readiness	MVP-ready with hardening required	Vercel config is present, but production secrets, refresh lifecycle, private files, and operational runbooks need work.

17. Glossary

Term	Meaning
Admin	User with role=admin; broad platform management and reporting access.
User	Normal application user with role=user.
Supervisor	A user with isSupervisor=true; can review work subject to project/user relationships.
Timesheet	Weekly project work record containing regular/overtime daily entries.
Project	Work container with client/SOW information, dates, manager, supervisor, and team.
Activity	Immutable-style event record used for timelines/auditing; current implementation does not enforce immutability.
Notification	User-facing event/message with read state and optional related resource.
Session	Server-side refresh-token record with expiry and MongoDB TTL cleanup.